Scaling Conjugate Gradients with MPI and OpenMP

Decomposition, communication hiding and memory bandwidth in a matrix-free 3D Poisson solver

WU YUHAN (A0351706H) ZUO DEWEI (A0352507H) ZHU CHEN (A0357394N)

DSA5208 · Project 3

Abstract

The Conjugate Gradient (CG) method is the standard solver for the symmetric positive definite systems produced by elliptic partial differential equations, and it is notoriously hard to scale: every iteration needs a nearest-neighbour exchange, two global reductions and a full sweep through memory. We build a matrix-free hybrid MPI + OpenMP CG solver for the 3D Poisson equation in which the four central parallelisation decisions—domain decomposition, halo exchange, reduction strategy and the split between processes and threads—are run-time switches, and measure each of them on the NUS Vanda cluster. The solver reproduces the serial solution to 3.8×10^{-15} in 180 parallel configurations and converges at the theoretical second order. On 36 cores of a Vanda node it reaches a speedup of 8.42 on a 512^3 grid and sustains 236.5 GB/s, 70% of the node's STREAM bandwidth: inside a node, CG scales exactly as far as memory bandwidth does. A memory-traffic model predicts the design trade-offs: pipelined CG pays its 36% traffic surcharge almost in full (1.32$\times$ measured) and leaves its true residual 69.4 times above the tolerance, slabs fall 13% behind blocks, overlapped halos only pay across a network, and the global reductions act mainly as barriers that turn bandwidth imbalance into waiting.

1. Introduction

Many simulation codes spend most of their run time solving a single kind of linear system. The pressure projection of incompressible flow solvers, the electrostatic potential in plasma and molecular dynamics, the gravitational potential in astrophysics and graph-Laplacian systems in semi-supervised learning and spectral clustering all reduce to a Poisson-type equation that has to be solved again at every time step or outer iteration. Discretising $-\nabla^2 u = f$ on a cube with N grid points per direction yields N^3 unknowns, and refining the grid makes the problem harder twice over: the system grows like N^3, and the number of iterations of a Krylov solver grows like N, so the total work grows like N^4. A 512^3 grid already holds 134 million unknowns and needs about two thousand CG iterations. Solving such problems in reasonable time and memory requires distributing them across many nodes.

The Conjugate Gradient method is the method of choice for these symmetric positive definite systems, and it is also a standard stress test for parallel computers: the HPCG benchmark [4], which complements LINPACK in the TOP500 ranking, is built around it because supercomputers reach only a few percent of their peak performance on it. The reason lies in the structure of a single iteration. Its matrix–vector product couples every unknown to its grid neighbours, so a distributed implementation has to exchange the boundary layers (*halos*) of every subdomain in every iteration. Its two inner products couple every unknown to every other one; each requires a global reduction that acts as a barrier across all processes and whose latency grows with the number of processes P. Since every iteration needs the scalars produced by the previous one, none of this communication can be batched or deferred across iterations. What remains after communication—the stencil and the vector updates—performs about 17 floating-point operations per 88 bytes of memory traffic, so even the perfectly parallel local work is limited by the memory bandwidth that all cores of a socket share.

Each of these obstacles admits several engineering answers, and the answers interact. A three-dimensional block decomposition minimises the halo volume but sends more, smaller and strided messages than a slab decomposition. Non-blocking communication can hide the halo exchange behind interior computation, but only if the MPI library actually progresses the messages and the deferred boundary work does not destroy cache locality. Algebraic reformulations of CG merge the two reductions into one or overlap the reduction with the matrix–vector product, at the price of extra vectors, extra memory traffic and weaker numerical stability. OpenMP threads inside each MPI process cut the number of messages and of reduction participants, at the price of fork–join overhead and threads that idle while the master thread communicates.

This project implements a serial reference solver and a hybrid MPI + OpenMP solver in C in which each of these choices is a run-time switch, so that its effect is measured in isolation on an identical problem. We validate the parallel solver point by point against the serial one in 180 configurations and against an analytical solution, derive a performance model from memory traffic and communication volume, and instrument the solver with per-phase timers and byte counters that explain the measured scaling rather than merely report it. Section 2 introduces the

problem and the algorithm, section 3 develops the parallel design and the performance model, section 4 validates the implementation and section 5 analyses its performance.

2. The model problem and the Conjugate Gradient method

We solve the Poisson equation $-\nabla^2 u = f$ on the unit cube $\Omega = (0,1)^3$ for an unknown function $u : \Omega \to \mathbb{R}$ and a given source f, with the homogeneous Dirichlet condition $u = 0$ on the boundary $\partial\Omega$. A uniform grid with N interior points per direction has spacing $h = 1/(N+1)$ and grid points $(x_i, y_j, z_k) = (ih, jh, kh)$ for $i, j, k \in \{1, \dots, N\}$; we write u_{ijk} for the approximation of $u(x_i, y_j, z_k)$. Replacing each second derivative by the centred difference $(u_{i-1} - 2u_i + u_{i+1})/h^2$ and multiplying by h^2 gives the linear system $A\mathbf{u} = \mathbf{b}$ with

$$\begin{aligned} (A\mathbf{u})_{ijk} &= 6u_{ijk} - u_{i-1,j,k} - u_{i+1,j,k} - u_{i,j-1,k} - u_{i,j+1,k} - u_{i,j,k-1} - u_{i,j,k+1}, \\ b_{ijk} &= h^2 f(x_i, y_j, z_k), \end{aligned} \tag{1}$$

where values with an index equal to 0 or $N+1$ are boundary values and therefore zero. The matrix $A \in \mathbb{R}^{N^3 \times N^3}$ is symmetric positive definite with condition number $\kappa(A) \approx 4/(\pi^2 h^2)$. We never assemble A: it is applied as the 7-point stencil (1) (*matrix-free*), so memory consumption and memory traffic are entirely those of the vectors.

To test the solver we prescribe the exact solution $u(x,y,z) = g(x)\,g(y)\,g(z)$ with $g(t) = t(1-t)e^t$, which vanishes on $\partial\Omega$, and compute the matching source $f = -\nabla^2 u$ analytically using $g''(t) = -t(3+t)e^t$. The common choice $\sin(\pi x)\sin(\pi y)\sin(\pi z)$ would be useless here: it is an eigenvector of the discrete operator, CG converges on it in a single iteration, and the test would hide most bugs. For our u the discrete solution differs from the exact one by a genuine $O(h^2)$ error, which turns the comparison with u into a sharp end-to-end test.

CG [1,6] (algorithm 1) builds the iterates $\mathbf{x}_k$ from the residuals $\mathbf{r}_k = \mathbf{b} - A\mathbf{x}_k$ and the search directions $\mathbf{p}_k$; the step length α_k minimises the A-norm of the error along $\mathbf{p}_k$, and β_{k+1} keeps the new direction A-conjugate to all previous ones. The scalar $\gamma_k = \|\mathbf{r}_k\|^2$ doubles as the stopping criterion with relative tolerance ε. The error contracts by roughly $(\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)$ per iteration, so the number of iterations to reach a fixed ε grows like $\sqrt{\kappa} \propto 1/h \propto N$. Our runs confirm this: 146 iterations for $N = 40$ and 1942 for $N = 512$ at $\varepsilon = 10^{-8}$, close to the ratio of the grid sizes. One iteration consists of one stencil application, two inner products and three vector updates. Per grid point the stencil costs 7 floating-point operations, its inner product 2 and the updates 8, i.e. 17 flops, against 11 loads and stores of 8-byte values (88 bytes, see section 3.5). At 0.2 flop/byte this is an order of magnitude below the balance of current server processors, so CG runs at the speed of memory, not of arithmetic.

Algorithm 1 Conjugate Gradients for $A\mathbf{x} = \mathbf{b}$ with $\mathbf{x}_0 = 0$. Framed comments mark the points where a parallel implementation must communicate.

1: $\mathbf{r}_0 = \mathbf{b}, \quad \mathbf{p}_0 = \mathbf{r}_0, \quad \gamma_0 = \mathbf{r}_0^T \mathbf{r}_0$
2: **for** $k = 0, 1, 2, \dots$ **until** $\sqrt{\gamma_k} \le \varepsilon \, \|\mathbf{b}\|$ **do**
3: $\quad \mathbf{q}_k = A\mathbf{p}_k$ ▷ halo exchange of $\mathbf{p}_k$, then stencil
4: $\quad \alpha_k = \gamma_k / (\mathbf{p}_k^T \mathbf{q}_k)$ ▷ global reduction
5: $\quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k, \;\; \mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{q}_k, \;\; \gamma_{k+1} = \mathbf{r}_{k+1}^T \mathbf{r}_{k+1}$ ▷ global reduction
6: $\quad \beta_{k+1} = \gamma_{k+1}/\gamma_k, \quad \mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_{k+1}\mathbf{p}_k$
7: **end for**

3. Parallel design

The parallel solver distributes the grid over P MPI processes and runs T OpenMP threads inside each. This section follows the data through one iteration: how the grid is partitioned, how boundary values travel between processes, how the global reductions are organised and how the threads share the local work. It closes with a cost model that ties these decisions together.

3.1 Data distribution

The processes form a Cartesian grid $p_x \times p_y \times p_z = P$ created by `MPI_Dims_create` and `MPI_Cart_create`. The process with coordinates (c_x, c_y, c_z) owns a block of $n_x \times n_y \times n_z$ grid points, where in each direction $n_d = \lfloor N/p_d \rfloor$, plus one for the first $N \bmod p_d$ coordinates; block sizes therefore differ by at most one plane and any P works with any N. Each block is stored with one ghost layer on every side, $(n_x+2)(n_y+2)(n_z+2)$ values with the k index contiguous. Ghost layers on the physical boundary permanently hold the Dirichlet zeros, so the stencil loop contains no branches. The right-hand side is evaluated from one-dimensional tables of g and g'', so no global vector $\mathbf{b}$ is stored.

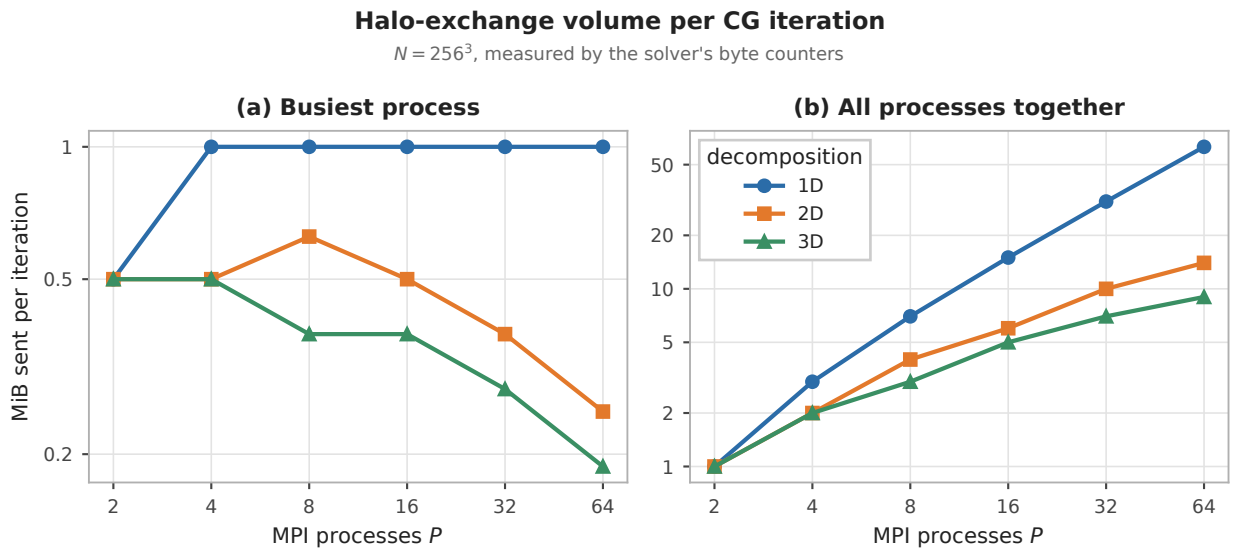


Figure 1: Halo data sent per CG iteration for slab (1D), pencil (2D) and block (3D) decompositions, $N = 256$, measured by the solver's counters. (a) The busiest process, which sets the exchange time on a bandwidth-limited network; (b) the total network load.

The switch `-d` selects slabs ($p_y = p_z = 1$), pencils ($p_z = 1$) or blocks (all three factors free). The choice sets how much data each process sends per iteration. For a process in the interior of the process grid and N divisible by the factors, the number of 8-byte values sent per halo exchange is

$$V_{1D} = 2N^2, \qquad V_{2D} = \frac{4N^2}{\sqrt{P}}, \qquad V_{3D} = \frac{6N^2}{P^{2/3}}, \tag{2}$$

carried by $m = 2$, 4 and 6 messages respectively. The slab volume does not decrease with P, so slabs stop scaling once the halo time becomes comparable to the computation, which shrinks like $1/P$; slabs also cap P at N. Blocks send the least data, but their k-faces consist of $n_x n_y$ values separated by a stride of $n_z + 2$, so packing them into a send buffer touches one cache line per value. The solver counts the bytes and messages of every process, and fig. 1 shows the measured values for $N = 256$. At $P = 64$ the busiest process sends 1.00 MiB per iteration with slabs and 0.19 MiB with blocks, a factor of 5.3, while the message count rises from 2 to 6. We use blocks as the baseline.

3.2 Halo exchange

Before each stencil application a process needs the values of the neighbouring blocks adjacent to its faces. Because the 7-point stencil uses no edge or corner neighbours, the three directions are independent and are exchanged together. In the *blocking* mode the solver packs all six faces into buffers within one OpenMP parallel region, performs one `MPI_Sendrecv` per direction and sense, unpacks the received faces into the ghost layers and then applies the stencil to the whole block. Neighbours that do not exist are `MPI_PROC_NULL`, which turns the corresponding transfers into no-ops.

The *overlapped* mode exploits the fact that most points of a block do not need ghost values. It posts `MPI_Irecv` for all faces, packs, posts `MPI_Isend`, applies the stencil to the interior, waits for the messages, unpacks and finally applies the stencil to the remaining boundary shell. The shell is covered by disjoint pieces—two i-planes, two j-rows inside them, two k-columns inside those—so the fused inner product counts every point exactly once. Making this overlap effective took two further steps. The first concerns progress: most MPI libraries, Open MPI over TCP included, move data only inside MPI calls, and large faces use a rendezvous protocol whose handshake cannot complete while the process computes. The solver therefore splits the interior sweep into chunks of i-planes (four by default) and calls `MPI_Testall` between them, which advances the transfers during the computation. The second concerns locality. Our first version deferred the shell layers on all six sides, so with slabs every process handled its $k = 1$ and $k = n_z$ columns as isolated, strided points even though these lie on the physical boundary and need no communication. This cost 10–15% per iteration in our measurements and made the overlapped mode slower than the blocking one. Restricting the deferred shell to layers that face an actual neighbour removed this penalty; what remains is the cost of the second pass over the shell, which pays off only when messages take real time (section 5.2).

Table 1: Cost of the three CG formulations per iteration.

variant	reductions ρ	values each	vectors	B_v (B/point)
classic CG	2	1	4	88
Chronopoulos–Gear (CG1)	1	2	5	88
pipelined CG	0 (overlapped)	2	7	120

3.3 Global reductions and three formulations of CG

Classic CG (algorithm 1) performs two `MPI_Allreduce` operations per iteration on a single value each. Nothing useful can run while they are in flight, and with recursive doubling each costs about $\lceil \log_2 P \rceil$ network latencies. Two algebraically equivalent reformulations attack this term.

The Chronopoulos–Gear variant [2] (`-a cg1`) introduces the auxiliary vectors $\mathbf{w}_k = A\mathbf{r}_k$ and $\mathbf{s}_k = A\mathbf{p}_k$. Since $\mathbf{p}_k = \mathbf{r}_k + \beta_k \mathbf{p}_{k-1}$, linearity gives the recurrence $\mathbf{s}_k = \mathbf{w}_k + \beta_k \mathbf{s}_{k-1}$, and the only stencil application per iteration becomes $\mathbf{w}_k = A\mathbf{r}_k$. The denominator $\mathbf{p}_k^T A \mathbf{p}_k$ of α_k can then be expressed through $\gamma_k = \mathbf{r}_k^T \mathbf{r}_k$ and $\delta_k = \mathbf{w}_k^T \mathbf{r}_k$: expanding $\mathbf{p}_k^T A \mathbf{p}_k$ and using $\mathbf{r}_k^T A \mathbf{p}_{k-1} = -\gamma_k / \alpha_{k-1}$ and $\mathbf{p}_{k-1}^T A \mathbf{p}_{k-1} = \gamma_{k-1} / \alpha_{k-1}$ yields

$$\beta_k = \frac{\gamma_k}{\gamma_{k-1}}, \qquad \alpha_k = \frac{\gamma_k}{\delta_k - \beta_k \gamma_k / \alpha_{k-1}}. \tag{3}$$

Both inner products are now available at the same moment and travel in one `MPI_Allreduce` of two values. The four vector updates ($\mathbf{p}, \mathbf{s}, \mathbf{x}, \mathbf{r}$) and γ are fused into one sweep and δ into the stencil sweep, so the memory traffic per iteration stays that of classic CG.

The pipelined variant of Ghysels and Vanroose [3] (`-a pipecg`) goes one step further and removes the dependency between the reduction and the stencil. It adds $\mathbf{z}_k = A\mathbf{s}_k$ and $\mathbf{q}_k = A\mathbf{w}_k$ with the recurrences $\mathbf{z}_k = \mathbf{q}_k + \beta_k \mathbf{z}_{k-1}$ and $\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha_k \mathbf{z}_k$. Now γ_k and δ_k depend only on $\mathbf{r}_k$ and $\mathbf{w}_k$, which are known before $\mathbf{q}_k$ is computed. The solver therefore starts an `MPI_Iallreduce` of (γ_k, δ_k), computes $\mathbf{q}_k = A\mathbf{w}_k$ while the reduction is in flight (polling it between stencil chunks, as for the halo), waits for the result, evaluates (3) and performs the six vector recurrences in one fused sweep that also produces the next local γ and δ. The reduction latency disappears behind the stencil. The price is three extra vectors, 36% more memory traffic, and recurrences for $\mathbf{w}$ and $\mathbf{z}$ whose rounding errors are never corrected, which limits the attainable accuracy (section 4).

3.4 Threads within a process

MPI is initialised with `MPI_THREAD_FUNNELED`: only the master thread calls MPI, always outside parallel regions. Every loop nest is an `omp parallel for collapse(2) schedule(static)` over the (i, j) index pairs with an `omp simd` loop over the contiguous k index, and the fused inner products are OpenMP reductions. Arrays are 64-byte aligned and initialised with the same static schedule that later computes on them, so on multi-socket nodes the first-touch policy places each thread's pages in its own NUMA domain. Processes and threads are pinned (`--map-by ppr:`n`:node:PE=`T `--bind-to core`, `OMP_PLACES=cores`, `OMP_PROC_BIND=close`), so the threads of one process share a socket. Fewer processes with more threads reduce the number of messages, the number of reduction participants and the ghost memory; in exchange every kernel pays a fork–join (five parallel regions per iteration with blocking halos, up to about fifteen with overlap), and the worker threads idle while the master thread waits in MPI.

3.5 Performance model

The decisions above enter the time per iteration through four terms,

$$t_{\text{iter}}(P) \;\approx\; \frac{N^3}{P}\,\frac{B_v}{b(P)} \;+\; m\,t_s + V\,t_w \;+\; \rho\,\lceil \log_2 P \rceil\, t_s, \tag{4}$$

where B_v is the memory traffic per grid point of the CG variant (table 1), $b(P)$ the memory bandwidth available to one process when P processes share the hardware, m and V the message count and halo volume of the busiest process from (2), t_s the latency of a message, t_w the transfer time per value, and ρ the number of reductions per iteration that are not hidden. B_v counts explicit loads and stores in the convention of the STREAM benchmark [5], assuming that the stencil's neighbour values are reused from cache. For classic CG the stencil reads $\mathbf{p}$ and writes $\mathbf{q}$, the $\mathbf{x}, \mathbf{r}$ update reads four vectors and writes two, and the $\mathbf{p}$ update reads two and writes one: $2 + 6 + 3 = 11$ accesses, or 88 bytes. Fusing the two inner products into these sweeps avoids three further reads.

The model makes the scaling behaviour explicit. The memory term falls like $1/P$ only while $b(P)$ stays constant, that is, until the processes on a socket saturate its memory bandwidth; from then on adding cores on the same

node gains nothing, while adding nodes adds bandwidth. The halo term falls like $P^{-2/3}$ for blocks and not at all for slabs. The reduction term grows with $\log_2 P$ and is the only term that grows, so it eventually dominates every strong-scaling run. CG1 halves it at no cost in B_v, and pipelined CG removes it at a 36% higher B_v, which pays off once the hidden latency exceeds the extra memory time.

4. Validation

We validate the solver at three levels: agreement with the serial code, convergence to the exact solution, and attainable accuracy. The script `scripts/validate.sh full` runs all tests and `scripts/check_validation.py` evaluates them; all of them pass.

The first level compares every parallel configuration with the serial solver point by point. We ran all combinations of the three CG variants, the two halo modes and the three decompositions on $P \in \{1, 2, 3, 4, 6, 8\}$ processes with $T \in \{1, 2\}$ threads ($T = 2$ for $P \leq 4$) on a 40^3 grid, chosen because 40 is not divisible by 3 or 6 so that uneven blocks occur. Each run writes its solution in global order with collective MPI-IO (`MPI_Type_create_subarray` and `MPI_File_write_all`), and the file is compared with the serial solution. Table 2 summarises the 180 runs. Classic CG and CG1 agree with the serial code to 3.8×10^{-15} relative to the solution's maximum, which is rounding level and stems only from the different summation order of the distributed inner products, and they need exactly the serial number of iterations (146). Pipelined CG agrees to 2.0×10^{-12}, the effect of its extra recurrences and still seven orders of magnitude below the discretisation error (2.39×10^{-5}). A further 30 runs cover the corner cases of the shell decomposition of section 3.2—one-plane slabs, two-point blocks, $N = 7$ on six processes and $N = 2$ on two—and all reproduce the serial solution to rounding level. The quick validation suite was repeated inside the Vanda job before the benchmarks (36 variant runs, 12 edge cases and the grid study) and passes with the same worst differences, 3.8×10^{-15} and 2.0×10^{-12}.

Table 2: Parallel against serial solution on the 40^3 grid. "max diff" is $\max |x_{\text{par}} - x_{\text{ser}}| / \max |x_{\text{ser}}|$ over all runs of a row; "true residual" is the largest $\|\mathbf{b} - A\mathbf{x}\|/\|\mathbf{b}\|$ at tolerance $\varepsilon = 10^{-8}$.

algorithm	halo	runs	P	T	iterations	max diff	true residual
CG	blocking	30	1–8	1–2	146–146	3.5×10^{-15}	8.24×10^{-9}
CG	overlap	30	1–8	1–2	146–146	3.1×10^{-15}	8.24×10^{-9}
CG1	blocking	30	1–8	1–2	146–146	3.8×10^{-15}	8.24×10^{-9}
CG1	overlap	30	1–8	1–2	146–146	3.8×10^{-15}	8.24×10^{-9}
pipelined CG	blocking	30	1–8	1–2	146–146	2.0×10^{-12}	8.24×10^{-9}
pipelined CG	overlap	30	1–8	1–2	146–146	2.0×10^{-12}	8.24×10^{-9}

The second level checks that the solver computes the solution of the differential equation and not only of some linear system. Table 3 and fig. 2(a) report the discrete L^2 error $\|u_h - u\|_{L^2} = \left(h^3 \sum_{ijk} (u_{ijk} - u(x_i, y_j, z_k))^2\right)^{1/2}$ for $N = 16$ to 128, solved to $\varepsilon = 10^{-10}$ so that the algebraic error is negligible. The observed order is 2.000, exactly the order of the 7-point scheme, and the parallel errors match the serial ones in every printed digit.

Table 3: Grid convergence of the serial solver and of the parallel solver ($P = 4$, blocks, overlapped halo).

N	h	iterations	$\|u_h - u\|_{L^2}$ serial	$\|u_h - u\|_{L^2}$ parallel	observed order
16	5.88×10^{-2}	68	5.210×10^{-5}	5.210×10^{-5}	–
24	4.00×10^{-2}	102	2.410×10^{-5}	2.410×10^{-5}	1.999
32	3.03×10^{-2}	136	1.383×10^{-5}	1.383×10^{-5}	2.000
48	2.04×10^{-2}	205	6.274×10^{-6}	6.274×10^{-6}	2.000
64	1.54×10^{-2}	274	3.565×10^{-6}	3.565×10^{-6}	2.000
96	1.03×10^{-2}	413	1.601×10^{-6}	1.601×10^{-6}	2.000
128	7.75×10^{-3}	552	9.052×10^{-7}	9.052×10^{-7}	2.000

The third level probes the numerical price of the reformulations. Figure 2(b) runs the three variants towards a tolerance of 10^{-15} and records both the recursively updated residual, which the algorithm sees, and the true residual $\|\mathbf{b} - A\mathbf{x}_k\|$, computed with an extra untimed stencil application. CG and CG1 are indistinguishable, and their true residual levels off at 6×10^{-13} once rounding errors in $\mathbf{x}$ dominate. Pipelined CG follows them until the two residuals separate near 3×10^{-11}; its best true residual is 1×10^{-11}, 15 times worse than CG, and when the iteration continues its recursive residual stagnates while the true residual drifts back up to 4×10^{-9} after 700 iterations. This is the residual gap of pipelined Krylov methods: latency hiding costs numerical robustness. On the 40^3 validation grid all three variants still meet $\varepsilon = 10^{-8}$ (table 2), but the gap grows with the problem: on the 512^3 grid of section 5.4

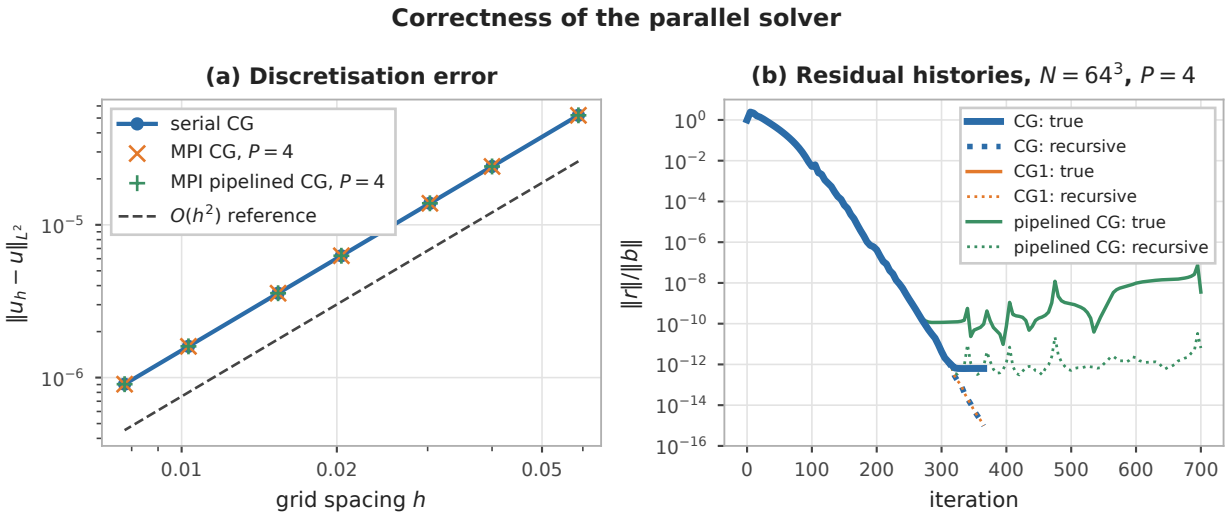


Figure 2: (a) Discretisation error against grid spacing for the serial and parallel solvers, with an $O(h^2)$ reference line. (b) True (solid) and recursively updated (dotted) relative residuals, $N = 64$, $P = 4$; the CG and CG1 curves coincide.

the true residual of pipelined CG ends at 69.4 times the tolerance its recursive residual reports. Production use of the pipelined variant therefore needs residual replacement, i.e. periodically recomputing $\mathbf{r} = \mathbf{b} - A\mathbf{x}$ and $\mathbf{w} = A\mathbf{r}$ explicitly.

5. Performance

Setup. All measurements in this section were taken on one compute node of the NUS Vanda cluster, submitted through PBS Pro to the `auto_free` queue. The node carries two Intel(R) Xeon(R) Gold 6438N processors with 32 cores each, 120 MB of last-level cache and two NUMA domains; the job owned 36 of its 64 cores, so its processes span both sockets. The code was built with GCC 13.1.0 (`-O3 -march=native -fopenmp`) and run with Open MPI 4.1.8, every process pinned to its own cores. Strong-scaling runs use $N = 512$, i.e. 134.2 million unknowns and four to seven vectors of 1.07 GB each, more than thirty times the cache. Each timing run performs exactly 50 iterations of the steady-state loop: work vectors are allocated before the clock starts, and the one-off setup stencil application of CG1 and pipelined CG counts in the time to solution (table 7) but not in the time per iteration. Every configuration was run 3 times, interleaved with the others, and the fastest run is reported; the median spread between repetitions was 0.2%, so differences of a few percent are real. Speedups refer to the serial code; the parallel code on one process is 6.3% slower than the serial code, the price of its halo machinery and OpenMP regions.

5.1 Strong scaling and the memory-bandwidth wall

Figure 3 and table 4 show the baseline: pure MPI, blocks, overlapped halo and classic CG. A serial iteration takes 459.0 ms. Up to four processes the speedup is close to ideal, 3.67 on four processes at 92% efficiency. From eight processes on it flattens: 5.8 on eight, 7.2 on sixteen and 8.42 on 36 processes, a parallel efficiency of 23%. Communication volume does not explain this. The busiest process sends 1.1 MiB per iteration at $P = 36$, and halo exchange plus reductions stay below 8% of the iteration up to $P = 32$. The limit is memory bandwidth.

Figure 4 converts the measured time per iteration into DRAM traffic with the 96-byte model of section 3.5 (88 bytes of explicit loads and stores plus the write-allocate of the stencil output) and compares it with the STREAM triad counted the same way. One core moves 26.4 GB/s, more than STREAM on that core (22.2 GB/s). STREAM itself grows linearly up to eight threads and then saturates: from 16 to 36 threads it gains only 21%. The solver follows the same curve and reaches 236.5 GB/s on 36 cores, 70% of the node's STREAM bandwidth (335.2 GB/s). Inside a node the speedup of CG is the bandwidth speedup; cores added after the memory controllers saturate have nothing to compute on. The kernel fusions of section 3.5 are correspondingly decisive, because every memory access they remove is a direct reduction in run time.

The breakdown in fig. 6 shows where the bandwidth goes. The vector updates take 58% of a single-process iteration, more than the stencil, because they account for nine of the eleven memory accesses per point while doing almost no arithmetic. Through $P = 32$ the communication phases stay thin. At $P = 36$ the picture changes: the reductions absorb 30 ms of a 54.5 ms iteration. This is waiting, not data movement, since an `MPI_Allreduce` of one value

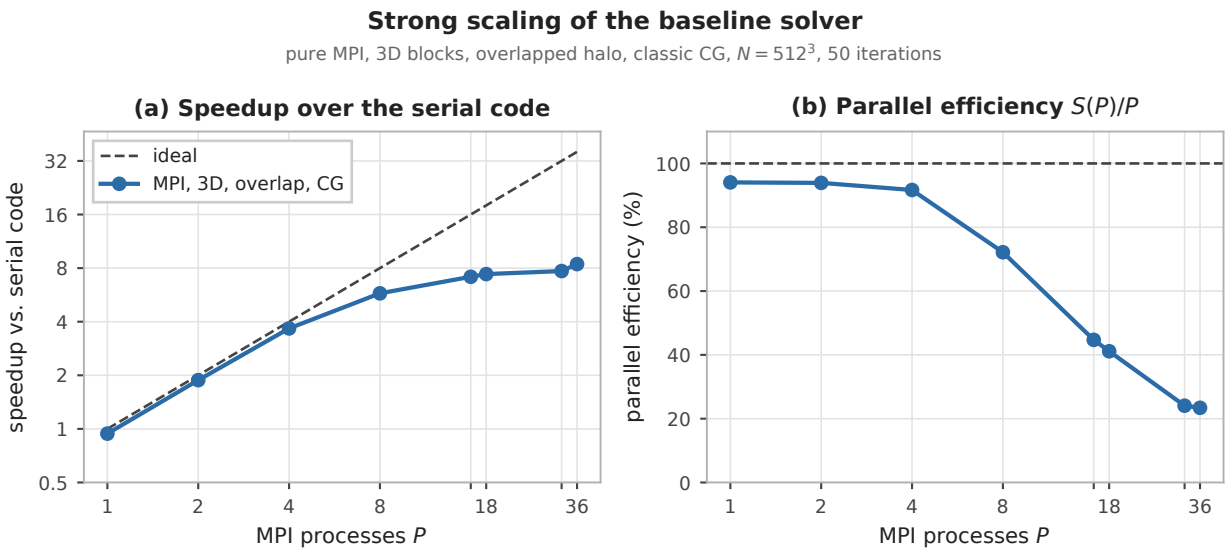


Figure 3: Strong scaling of the baseline (pure MPI, 3D blocks, overlapped halo, classic CG), $N = 512^3$, on one Vanda node.

Table 4: Strong scaling, $N = 512^3$, 50 iterations. "comm" is the share of an iteration spent in halo exchange and reductions, including the time processes wait there for slower ones; MLUP/s is millions of grid-point updates per second.

P	process grid	ms / iteration	speedup	efficiency (%)	comm (%)	MLUP/s
1	1×1×1	487.98	0.94	94	0.0	275
2	2×1×1	244.38	1.88	94	2.3	549
4	2×2×1	125.16	3.67	92	4.8	1072
8	2×2×2	79.51	5.77	72	7.4	1688
16	4×2×2	64.17	7.15	45	7.6	2092
18	3×3×2	62.00	7.40	41	5.3	2165
32	4×4×2	59.60	7.70	24	5.0	2252
36	4×3×3	54.48	8.42	23	71.1	2464

takes microseconds inside a node. The job's 36 cores necessarily span both 32-core sockets, unevenly, so the processes on the less crowded socket get more memory bandwidth, finish their sweeps early and wait at the next reduction for the processes on the crowded one. The global reductions turn bandwidth imbalance into idle time, exactly the barrier effect described in section 1.

5.2 Effect of the design choices

Figure 7 and table 5 vary one decision at a time around the baseline and report the time per iteration relative to the baseline at the same P. Three results stand out.

The CG formulations follow the traffic model quantitatively. Pipelined CG moves 120 instead of 88 bytes per point, a ratio of 1.36. Its measured cost rises from 1.11 times the baseline on four processes to 1.32 on 36 processes and converges to that ratio as the node becomes bandwidth-bound: on a single node the reduction latency it hides is worth microseconds, so only the extra traffic remains. CG1 is 10% faster than the baseline on four processes and equal from 16 processes on (1.01 at $P = 36$). It moves the same bytes, and its advantage comes from the single fused update sweep, which counts while the cores rather than the memory set the pace.

The decompositions follow the volume model of fig. 1. Up to eight processes slabs are within 2% of blocks. From 16 processes on, slabs fall behind and run 13–14% slower at $P = 32$ and $P = 36$, where every process ships two full 512^2 planes, 4 MiB per iteration against 1.1 MiB for blocks. Pencils are never slower than blocks and up to 5% faster: they avoid the strided k-faces, whose gather and shell pass cost more on one node than the extra halo volume saves.

Overlapping the halo exchange does not pay on one node. The blocking exchange is 2–7% faster at every P (0.97 at $P = 36$). Messages between processes on the same node are memory copies that complete in microseconds, so there is nothing to hide, while the overlapped mode pays for the second pass over the boundary shell and for the

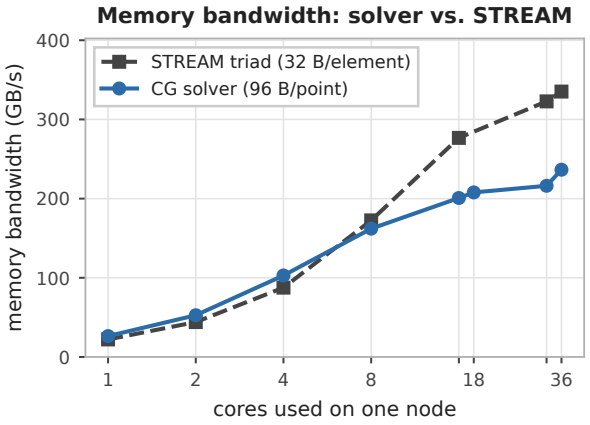


Figure 4: DRAM traffic rate of the CG baseline (96 bytes per point and iteration) against the STREAM triad (32 bytes per element), both including write-allocate traffic.

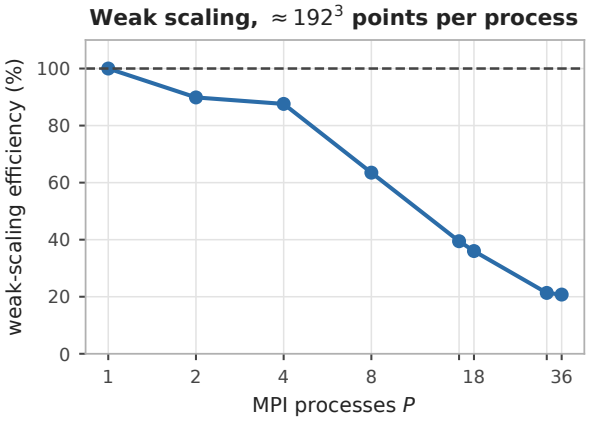


Figure 5: Weak scaling: time per grid point and iteration relative to one process.

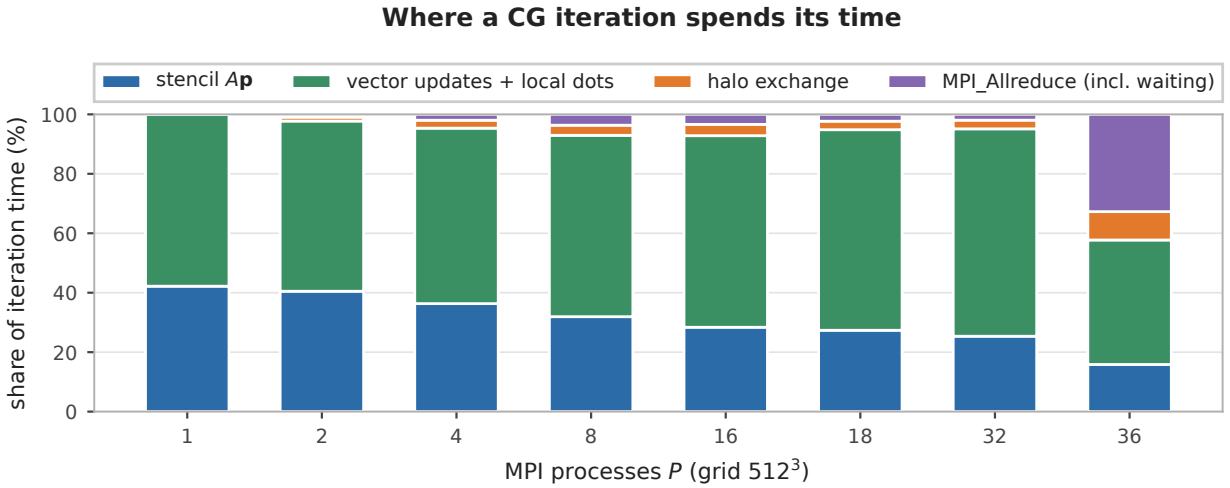


Figure 6: Share of the iteration time spent in the stencil, the vector updates, the halo exchange and the reductions (each phase timed as the maximum over processes; the reduction time includes waiting for slower processes; absolute times in table 4).

polling. Overlap is built for a network, where each message costs tens of microseconds.

5.3 Processes versus threads

Figure 8 and table 6 keep all 36 cores busy and vary the split into MPI processes and OpenMP threads. The fastest split is 9×4, 1.03× faster than pure MPI. Two or four threads per process are best (52.8 and 52.7 ms): fewer processes mean fewer and larger halo messages and fewer participants in each reduction. From six threads per process on, the reduction wait disappears (below 1 ms per iteration), but the iteration does not get faster. The imbalance has moved inside the processes, where threads that share a slow memory controller now hold up the OpenMP barrier at the end of each sweep instead of the `MPI_Allreduce`. With more threads the fork–join cost grows, and a single process with 36 threads is 16% slower than the best split. The right configuration is a few processes per socket with a handful of threads each.

5.4 Weak scaling and time to solution

For weak scaling each process keeps about 192^3 grid points, i.e. $N = \mathrm{round}(192\,P^{1/3})$. Figure 5 reports the time per grid point and iteration relative to one process. At $P = 36$ the efficiency is 21%: one process completes its iteration in 21.1 ms, 36 processes need 102.9 ms for 36 times the data. The cause is the same wall: the bandwidth available to each process drops from 32 GB/s to 6.6 GB/s. On one node, weak and strong scaling measure the same thing, how the memory bandwidth is shared. Across nodes, where every node brings its own memory, weak scaling isolates the halo and reduction terms of (4). Weak scaling of the time per iteration also flatters the algorithm: the

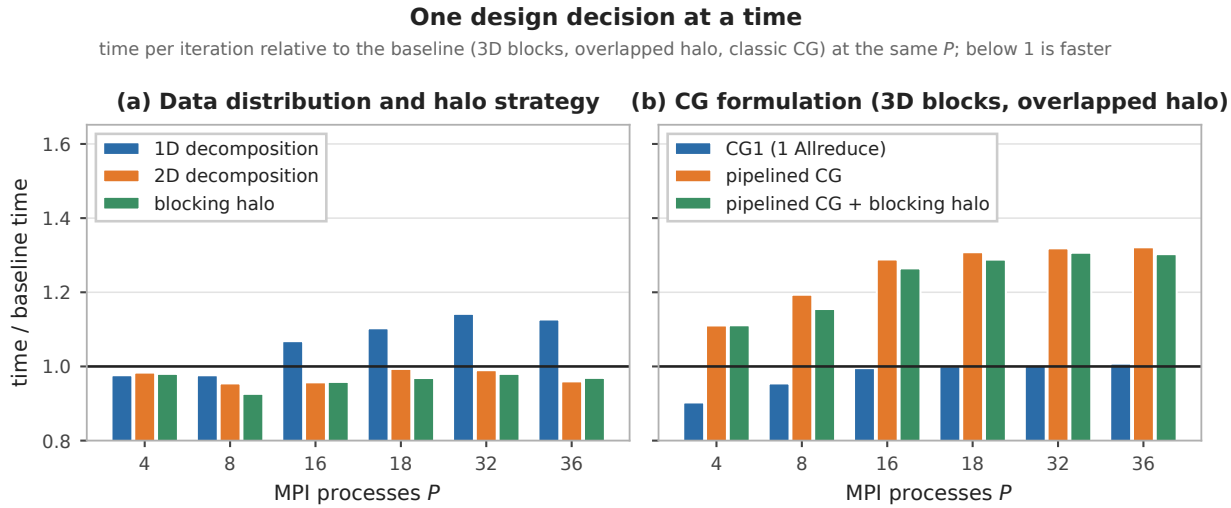


Figure 7: Time per iteration of each variant divided by the baseline (3D blocks, overlapped halo, classic CG) at the same P; values below 1 are faster.

Table 5: Time per iteration in ms and, in parentheses, relative to the baseline; $N = 512^3$ (four of the process counts; fig. 7 shows all of them).

variant	$P = 4$	$P = 16$	$P = 18$	$P = 36$
3D, overlap, CG (baseline)	125.16	64.17	62.00	54.48
1D decomposition	122.21 (0.98)	68.55 (1.07)	68.39 (1.10)	61.38 (1.13)
2D decomposition	123.10 (0.98)	61.43 (0.96)	61.59 (0.99)	52.29 (0.96)
blocking halo	122.66 (0.98)	61.50 (0.96)	60.07 (0.97)	52.80 (0.97)
CG1 (1 Allreduce)	113.02 (0.90)	63.89 (1.00)	62.04 (1.00)	54.90 (1.01)
pipelined CG	139.00 (1.11)	82.69 (1.29)	81.10 (1.31)	71.99 (1.32)
pipelined CG + blocking halo	139.07 (1.11)	81.15 (1.26)	79.87 (1.29)	70.98 (1.30)

iteration count grows like $N \propto P^{1/3}$, so the time to solution grows by the same factor even when every iteration scales perfectly, a growth that only a better preconditioner removes.

Table 7 closes the loop with complete solves to $\varepsilon = 10^{-8}$ on all 36 cores. All three variants stop after 1942 iterations and reach the same error against the exact solution, which at this tolerance is set by the discretisation; CG and CG1 finish in about 106 s, pipelined CG takes 140 s. The true residuals differ: CG and CG1 end at 9.8×10^{-9}, pipelined CG at 6.9×10^{-7}, 69.4 times the tolerance its recursive residual reports. On the 256^3 development grid the same ratio was 4; the residual gap grows with the problem and is the second cost of hiding the reduction, next to the extra memory traffic.

6. Conclusions

We built a matrix-free hybrid MPI + OpenMP Conjugate Gradient solver for the 3D Poisson equation in which decomposition, halo exchange, reduction strategy and threading are run-time switches. It reproduces the serial solution to rounding level in 180 configurations and 30 edge cases, and again in the quick suite run on Vanda, and converges to the exact solution at the theoretical second order, so every performance difference we measure is a property of the parallel design.

On a Vanda node CG runs into the memory-bandwidth wall. The solver scales almost perfectly to four processes, reaches a speedup of 8.42 on 36 cores and moves 236.5 GB/s at that point, 70% of the node's STREAM bandwidth; strong and weak scaling inside the node both reduce to how that bandwidth is shared. The memory-traffic model predicts the design trade-offs to within a few percent: pipelined CG pays its 36% traffic surcharge almost in full (1.32 measured), CG1 is up to 10% faster than classic CG and never more than 1% slower, slabs lose 13–14% once their constant halo volume stops shrinking, and overlapping the halo pays only where messages cross a network. The global reductions do more damage as barriers than as messages: on 36 cores they turn the bandwidth imbalance between two sockets into 30 ms of waiting per iteration, and two or four OpenMP threads per process recover 3% of the iteration.

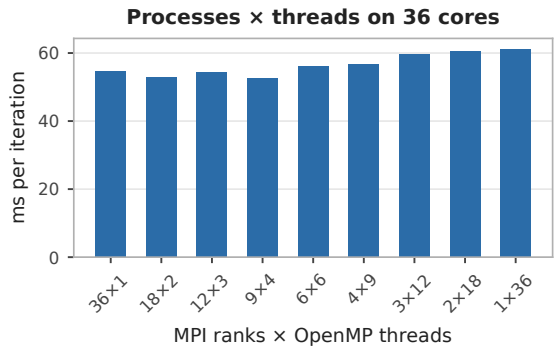


Figure 8: Time per iteration for different process × thread splits of the same 36 cores.

Table 6: Process × thread splits: time per iteration and its communication parts (ms, including waiting), and the total halo volume per iteration (MiB).

P	T	total	halo	reduce	MiB
36	1	54.48	9.83	30.30	28.00
18	2	52.78	7.96	28.23	20.00
12	3	54.25	6.71	27.90	16.00
9	4	52.68	10.83	22.34	16.00
6	6	55.98	1.46	0.68	12.00
4	9	56.76	0.83	0.75	8.00
3	12	59.54	1.25	0.55	8.00
2	18	60.45	1.31	0.35	4.00
1	36	61.23	0.13	0.00	0.00

Table 7: Complete solves to $\varepsilon = 10^{-8}$, $N = 512^3$, 36 cores, blocks, overlapped halo.

algorithm	P	iterations	time (s)	ms / iteration	true residual	max error
CG	36	1942	105.72	54.44	9.76×10^{-9}	1.53×10^{-7}
CG1	36	1942	106.77	54.97	9.76×10^{-9}	1.53×10^{-7}
pipelined CG	36	1942	139.58	71.87	6.94×10^{-7}	1.53×10^{-7}

For the reduction term the three CG formulations offer a clear trade-off. Merging the two reductions (Chronopoulos–Gear) costs nothing and is the formulation we recommend by default. Hiding the reduction completely (pipelined CG) costs 36% more memory traffic and leaves the true residual 69.4 times above the requested tolerance on the 512^3 grid, so it needs residual replacement and wins only across many nodes, where reduction latency exceeds the extra memory time. The natural next steps are a multigrid or Chebyshev preconditioner, which removes the $O(N)$ growth of the iteration count, and multi-node runs, where the communication design of section 3 comes into play.

Code and data. The source code, the PBS job scripts, the scripts that produce every figure and table, the raw CSV measurements and a `README.md` with full run instructions are open source (MIT licence) at https://github.com/davidzuo023/dsa5208-cg-poisson [7].

References

1. M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *J. Res. Nat. Bur. Standards* 49:409–436, 1952.
2. A. T. Chronopoulos and C. W. Gear. s-step iterative methods for symmetric linear systems. *J. Comput. Appl. Math.* 25:153–168, 1989.
3. P. Ghysels and W. Vanroose. Hiding global synchronization latency in the preconditioned Conjugate Gradient algorithm. *Parallel Computing* 40:224–238, 2014.
4. J. Dongarra, M. A. Heroux and P. Luszczek. High-performance conjugate-gradient benchmark: a new metric for ranking high-performance computing systems. *Int. J. High Perform. Comput. Appl.* 30(1):3–10, 2016.
5. J. D. McCalpin. Memory bandwidth and machine balance in current high performance computers. *IEEE TCCA Newsletter*, 1995.
6. J. R. Shewchuk. *An Introduction to the Conjugate Gradient Method Without the Agonizing Pain*. Carnegie Mellon University, 1994.
7. Y. Wu, D. Zuo and C. Zhu. *dsa5208-cg-poisson: hybrid MPI+OpenMP Conjugate Gradients for the 3D Poisson equation*. Source code, 2026. https://github.com/davidzuo023/dsa5208-cg-poisson.